**Evidencia: Integración de una API en el proyecto Flutter**

**1. Breve descripción del proyecto** La aplicación "Videojuegos Disada" es una plataforma diseñada para que los jugadores (gamers) registren sus perfiles y descubran nuevos títulos para jugar. Actualmente, la app permite registrar el nombre del jugador, su videojuego favorito y la plataforma que utiliza. Para enriquecer la experiencia, necesitamos mostrar un catálogo de videojuegos gratuitos disponibles en el mercado. Para obtener estos datos, utilizaremos una API pública de videojuegos que nos proporcionará información en tiempo real, como el título, el género y la plataforma de cada juego.

**2. Selección de una API**

- **URL de la API utilizada:** https://www.freetogame.com/api/games
- **Qué tipo de datos devuelve:** Devuelve una lista de objetos en formato JSON. Cada objeto representa un videojuego gratuito e incluye múltiples propiedades, de las cuales extraeremos el título (title - String), el género (genre - String) y la plataforma (platform - String).

**3. Clase modelo creada** Esta clase representa la estructura de los datos que recibiremos de la API.

```
class VideojuegoApi {
  // 1. Propiedades que guardarán los datos específicos que necesitamos de la API
  final String title;
  final String genre;
  final String platform;

  // 2. Constructor: Exige que estos tres datos sean proporcionados siempre que se cree un nuevo objeto VideojuegoApi
  VideojuegoApi({
    required this.title,
    required this.genre,
    required this.platform,
  });

  // 3. Factory method (Método de fábrica): Es como un "traductor".
  // Toma el formato JSON que llega de internet (que Dart ve como un Map o Diccionario)
  // y lo convierte en un objeto de tipo VideojuegoApi que nuestra app puede manejar fácilmente.
  factory VideojuegoApi.fromJson(Map<String, dynamic> json) {
    return VideojuegoApi(
      title: json['title'],       // Extrae el valor de la etiqueta 'title' del JSON
      genre: json['genre'],       // Extrae el valor de la etiqueta 'genre' del JSON
      platform: json['platform'], // Extrae el valor de la etiqueta 'platform' del JSON
    );
  }
}
```

**4. Clase del servicio API** Esta clase se encarga de realizar la petición HTTP y convertir el JSON en una lista de objetos de nuestro modelo.

```
1  import 'dart:convert'; // Importante: Nos permite usar json.decode() para interpretar el texto JSON
2  import 'package:http/http.dart' as http; // Paquete necesario para hacer peticiones web
3
4  class ApiService {
5    // Función asíncrona (async) porque va a tardar un tiempo en ir a internet y volver.
6    // Devuelve un 'Future' que eventualmente contendrá una Lista de nuestros objetos VideojuegoApi.
7    Future<List<VideojuegoApi>> obtenerJuegos() async {
8
9      // 1. Hacemos la petición GET a la URL exacta de la API
10     final response = await http.get(
11       Uri.parse('https://www.freetogame.com/api/games')
12     );
13
14     // 2. Verificamos el código de estado HTTP. 200 significa "Todo salió perfecto (OK)"
15     if (response.statusCode == 200) {
16
17       // 3. Convertimos el cuerpo de la respuesta (texto puro) en una lista dinámica manejable
18       List jsonResponse = json.decode(response.body);
19
20       // 4. Recorremos esa lista (.map). Por cada 'juego' en formato JSON que encontremos,
21       // usamos nuestro "traductor" (VideojuegoApi.fromJson) para convertirlo en un objeto.
22       // Finalmente, volvemos a empaquetar todo como una Lista real de Dart (.toList()).
23       return jsonResponse
24           .map((juego) => VideojuegoApi.fromJson(juego))
25           .toList();
26
27     } else {
28       // 5. Si el servidor responde con un error (ej. 404 No encontrado o 500 Error de servidor),
29       // lanzamos una excepción para que nuestra app no colapse silenciosamente.
30       throw Exception("Error al cargar los videojuegos");
31     }
32   }
33 }
```

**5. Mostrar los datos y personalizar la interfaz (Partes 5 y 6)** Este es el código de la pantalla que utiliza FutureBuilder para consumir el servicio y muestra la información utilizando un widget Card, iconos y espaciado.

```
import 'dart:convert'; // Importante: Nos permite usar json.decode() para interpretar el texto JSON
import 'package:http/http.dart' as http; // Paquete necesario para hacer peticiones web

class ApiService {
  // Función asíncrona (async) porque va a tardar un tiempo en ir a internet y volver.
  // Devuelve un 'Future' que eventualmente contendrá una Lista de nuestros objetos VideojuegoApi.
  Future<List<VideojuegoApi>> obtenerJuegos() async {

    // 1. Hacemos la petición GET a la URL exacta de la API
    final response = await http.get(
      Uri.parse('https://www.freetogame.com/api/games')
    );

    // 2. Verificamos el código de estado HTTP. 200 significa "todo salió perfecto (OK)"
    if (response.statusCode == 200) {

      // 3. Convertimos el cuerpo de la respuesta (texto puro) en una lista dinámica manejable
      List jsonResponse = json.decode(response.body);

      // 4. Recorremos esa lista (.map). Por cada 'juego' en formato JSON que encontremos,
      // usamos nuestro "traductor" (VideojuegoApi.fromJson) para convertirlo en un objeto.
      // Finalmente, volvemos a empaquetar todo como una Lista real de Dart (.toList()).
      return jsonResponse
          .map((juego) => VideojuegoApi.fromJson(juego))
          .toList();

    } else {
      // 5. Si el servidor responde con un error (ej. 404 No encontrado o 500 Error de servidor),
      // lanzamos una excepción para que nuestra app no colapse silenciosamente.
      throw Exception("Error al cargar los videojuegos");
    }
  }
}import 'package:flutter/material.dart';
// Nota: Asegúrate de importar tus archivos de ApiService y VideojuegoApi aquí arriba

// Usamos StatefulWidget en lugar de StatelessWidget porque esta pantalla va a cambiar
// de estado (de "Cargando..." a "Mostrar Lista") una vez que lleguen los datos.
class ListaJuegosPage extends StatefulWidget {
  @override
  ListaJuegosPageState createState() => ListaJuegosPageState();
}

class _ListaJuegosPageState extends State<ListaJuegosPage> {
  // Variable que va a "sostener" la promesa de que los datos llegarán en el futuro
  late Future<List<VideojuegoApi>> futureJuegos;

  @override
  void initState() {
    super.initState();
    // initState se ejecuta una sola vez cuando la pantalla se abre.
    // Aquí le decimos al servicio: "¡Ve a buscar los juegos ahora!"
    futureJuegos = ApiService().obtenerJuegos();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Catálogo de Juegos Gratuitos')),

      // FutureBuilder es el widget mágico que "escucha" a futureJuegos y reconstruye la pantalla
```